ONEOFFICEAUTOMATION: technical architecture

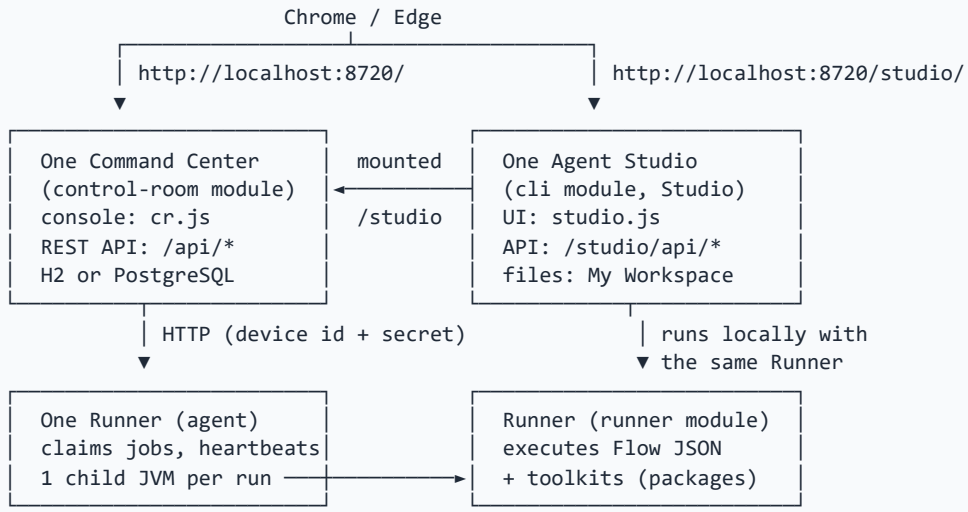
How the product is built, configured and managed, and how to build new parts of it. Status as of 20 Oct 2026 (commit `dd9fab5`). Paths are relative to the repository root `C:\OOA\oneofficeautomation`.

Contents

1. The product in one picture
2. Technology stack
3. Repository layout
4. Modules and how they depend on each other
5. Core concepts
6. Runtime flows
7. Data model (Command Center database)
8. Security model
9. Configuration
10. Managing the platform
11. Testing
12. How to build new components
13. Known gaps and planned work

---

1. The product in one picture



- **One Agent Studio** is where people build agents. It saves files in a private folder (My Workspace) and runs them on the same computer.
- **One Command Center** is the shared server. It holds the agent repository (versions and the live production version), users and roles, the vault, schedules, triggers, work queues, workflows, the audit trail and the dashboards.
- **One Runner** is the machine-side worker. It registers with the Command Center, asks for jobs, and runs each agent in its own Java process.
- The **Runner engine** (`runner` module) executes an agent definition (**Flow JSON**) step by step, calling **toolkits** (packages of actions).

During development, `ooa server` starts all four in **one Java process on one port (8720)**: the Command Center at `/`, Studio mounted at `/studio`, and a One Runner for this computer.

---

2. Technology stack

Layer	Technology	Why
Language	**Java 21** (toolchain and `--release 21`)	Records, pattern matching, switch expressions, virtual-thread-ready JDK
Build	**Gradle** (Kotlin DSL, wrapper `gradlew.bat`), version catalog `platform/gradle/libs.versions.toml`	Multi-module build, optional modules
HTTP servers	**JDK `com.sun.net.httpserver.HttpServer`**	No framework; small, fast start, easy to embed
HTTP client	**JDK `java.net.http.HttpClient`** (wrapped by `HttpJson`)	Every cloud toolkit uses it; no SDKs
JSON	Own `JsonLite` (sdk) for parsing and writing; Jackson 2.17 + networknt JSON Schema 1.4 (adapters-jackson) for schema validation	The core has no third-party JSON dependency
Database	**H2 2.2** embedded (default) or **PostgreSQL 42.7** driver	Plain JDBC, own pool and migrations (`Db.java`)
Office files	Apache POI 5.3	Excel, Word, PowerPoint
PDF	Apache PDFBox 3.0	
Email	Eclipse Angus Mail 2.0 (Jakarta Mail)	SMTP and IMAP
Browser	Microsoft Playwright for Java 1.47	Web automation and recorder
FTP/SFTP	Apache Commons Net 3.11, JSch (mwiede fork) 0.2	
Windows	JNA platform 5.14	Windows, DLL calls, Credential Manager, hotkeys
Logging	SLF4J + Logback 1.5 (runtime only, in cli)	
Crypto	JDK only: PBKDF2-HMAC-SHA256, AES-256-GCM, Ed25519, SHA-256, RSA/EC for OpenID Connect	No crypto libraries
Front ends	**Plain JavaScript, HTML and CSS**, no framework or build step	Served as static files from the JARs
Tests	JUnit 5.10 (Java); **jsdom** (Node.js) for the two browser UIs	

3. Repository layout

```
oneofficeautomation/
├─ platform/                the Java product (all modules below)
│  ├─ settings.gradle.kts   module list; optional modules come from gradle.properties
│  ├─ gradle.properties     ooa.optionalModules=packages-office,packages-email,...
│  ├─ build.gradle.kts      shared settings: Java 21, JUnit 5, test logging
│  ├─ scripts/start-ooa.bat one-click start (runs `ooa server`)
│  └─ <modules>/
├─ contracts/
│  ├─ flow-schema/flow.v1.json   JSON Schema of an agent definition (the "Flow")
│  └─ conformance/cases          shared test cases the runner must pass
├─ docs/                    guides, catalog, research, validation reports, this file
├─ examples/                sample agents (excel, refund)
└─ reference/               earlier Python reference implementation (design source; not shipped)
```

4. Modules and how they depend on each other

```
sdk  ◄── runner  ◄── packages  ◄── packages-office / -email / -pdf / -browser / -transfer / -windows
(optional)
                          ▲   ▲
                          │   └──────────── control-room   (Command Center)
                          ├──────────────── agent          (One Runner)
                          └──────────────── cli            (Studio + `ooa` command; also pulls in control-
room, agent,
                                                             adapters-jackson and the optional modules at
runtime)
runner ◄── adapters-jackson   (JSON Schema validation; loaded by name if present)
```

Module	Size (main code)	What it contains
`sdk`	~0.8k lines	The contract for toolkits: annotations `@OoaPackage` `@Action` `@Input` `@Output` `@Condition`, `ActionContext`, `OoaType`, `SecureString`, `OoaException`/`ActionException`, `PackageRegistry`, `PackageIntrospector` (builds manifests from annotations), `PackageSigning` (Ed25519), `JsonLite`, `LoopIterator`, `ChildAgents`
`runner`	~2k lines	`Runner` (the engine), `CorePackage` (if, loop, try, step, parallel, set, log, delay, wait_until, throw, comment, break, continue, run_bot), `expr/Expressions` + `Functions` (sandboxed expression language), `FlowValidator`, `Steps`, `RunEvent`, `RunResult`, interfaces `CredentialVault`, `AutomationRepository`, `BotInvoker`, `EventSink`
`packages`	63 toolkits	Built-in toolkits with no heavy dependencies: data (string, number, list, dictionary, datatable, datetime, json, xml, csv), files, system, REST/SOAP, database, crypto, PGP, OCR, image, desktop, AI (genai, ai_agent, ai_skills), cloud apps (Microsoft 365, Google, Salesforce, ServiceNow, Jira, Slack, Workday, GitHub, Azure DevOps, Asana, Zendesk, Okta, Box, DocuSign, Snowflake, Databricks, Kafka), SAP GUI, terminal, connectors, interrupts. Shared helpers: `HttpJson`, `LlmClient`, `ModelConnections`, `XmlSupport`, `ProcessRunner`, `FirstPartyPackages` (the registry list)
`packages-office`	3 toolkits	Excel, Word, PowerPoint (POI)
`packages-email`	1	Email (SMTP/IMAP)
`packages-pdf`	1	PDF
`packages-browser`	1	Browser automation and recorder (Playwright)
`packages-transfer`	1	FTP/FTPS/SFTP
`packages-windows`	3	Window, DLL, Credential Manager (JNA)
`control-room`	~4.6k lines	One Command Center: `ControlRoom` (core services), `Platform` (queues, pools, roles, settings, API keys, connections, calendars, toolkits, governance), `Workflows` (One Workflow + task inbox), `ControlRoomServer` (HTTP API and console), `Db` (JDBC + migrations), `Vault`, `ExternalVaults`, `Security`, `Rbac`, `Audit`, `Sso`, `McpServer`, `TriggerKinds`, `TriggerPollers` and mail/HTTP poll sources; console UI in `src/main/resources/console`

Module	Size (main code)	What it contains
`agent`	~0.7k lines	One Runner: `Agent` (register, heartbeat, claim jobs, relay events), `RunWorker` (child process that runs one agent), `AgentClient` (HTTP), `LocalTriggers` (file, window, process, service, hotkey watchers)
`cli`	~1.8k lines	`Ooa` (command line: server, studio, control-room, agent, run, validate, manifest, reset-admin), `StudioServer` (Studio HTTP API), `Workspace` (My Workspace files), `ControlRoomLink` (check-in/out to the Command Center), `StudioDebugger`, `TriggerService`, `EnvVault`; Studio UI in `src/main/resources/studio`
`adapters-jackson`	small	`JacksonJsonCodec`, `NetworkntFlowSchemaValidator`

The dependency rules are **enforced by a test** (`cli/src/test/.../ArchitectureTest.java`). It fails the build when these rules are broken:

- `sdk` imports nothing else.
- `runner` knows no toolkits, no Studio, no Command Center and no Runner service.
- Built-in `packages` never import Studio or the optional modules.
- Optional modules depend only on sdk, runner and built-in packages.
- Studio (`cli`) loads optional modules **by class name only**, so a missing module never breaks it.
- The Command Center imports neither Studio nor the One Runner.
- The One Runner talks to the Command Center **only over HTTP**.
- Programs are never started through a shell: no `cmd /c` or `sh -c`, argument lists only.
- XML is parsed only through `XmlSupport`, which blocks XXE.
- Command Center SQL uses `?` placeholders only.
- Toolkits never print to the console; they log through `ActionContext`, so secrets are masked.

---

5. Core concepts

5.1 Agent definition (Flow JSON)

Every agent is a JSON file (`*.flow.json`) that follows `contracts/flow-schema/flow.v1.json`. In outline:

```
{
  "kind": "TaskBot",                       // TaskBot | ApiTask (headless API agent) | Process | Form |
Agent
  "id": "invoice_worker", "name": "Invoice worker", "description": "...",
  "variables": [
    {"name": "work_item", "type": "Dictionary", "direction": "input", "required": true},
    {"name": "total",     "type": "Number",     "direction": "output"}
  ],
  "steps": [
    {"id": "s1", "action": "core/if", "args": {"condition": {"op": "eq", "left": "$work_item.kind",
"right": "bad"}},
     "children": [{"id": "s2", "action": "core/throw", "args": {"code": "BUSINESS.BAD_ITEM", "message":
"bad item"}}], "else": []},
    {"id": "s3", "action": "core/set", "args": {"variable": "total", "value": "$work_item.amount"}}
  ]
}
```

- **Steps** call an action by id: `"<toolkit id>/<action id>"`, for example `ooa.excel/open`.
  - `args` holds the inputs.
  - `out` maps the action's outputs to variables.
  - Optional fields: `retry`, `timeoutSeconds`, `continueOnError`, `disabled`.
- **Variable types:** String, Number (always `BigDecimal`), Boolean, DateTime, List, Dictionary, Table (`{"columns":[...],"rows":[[...]]}`), File, Credential (a `SecureString`) and Any.
- **Values in args:**
  - `"$name"` and `"$a.b[0]"` keep the value's type.
  - `"Hi ${name}"` interpolates text.
  - `{"expr": "max($x, 3)"}` evaluates an expression.
  - `{"$credential": "folder/credential/attribute"}` is a vault reference.
  - `$sys.` *and* `$global.` are read-only built-ins.
- Other item types live beside agents in the workspace:
  - forms `*.form.json`
  - workflows `*.process.json`
  - document intelligence `*.doc.json`
  - Prompt Skills `ai-skills/*.json`
  - scripts `.py`, `.ps1`, `.vbs`, `.js`
  - custom connectors `connectors/*.connector.json`

5.2 Toolkits (packages) and actions

A toolkit is a Java class with annotations. The registry builds its manifest (which drives Studio's action panels, the validator and the AI assistant) by reading the annotations:

```
@OoaPackage(id = "acme.crm", name = "CRM", version = "1.0.0", description = "Read and update CRM
records.")
public final class CrmPackage {
    @Action(id = "get_customer", name = "Get customer", description = "Find a customer by id.",
capabilities = "network")
    @Output(name = "customer", type = OoaType.DICTIONARY, description = "Customer record.")
    public Map<String, Object> getCustomer(ActionContext ctx,
            @Input(name = "customer_id", type = OoaType.STRING, description = "Customer id.") String id,
            @Input(name = "token", type = OoaType.CREDENTIAL, description = "API token from the vault.")
Object token) {
        // ctx.info(...), ctx.warn(...): log lines (secrets masked)
        // ctx.service("vault", ...), ctx.sessions(), ctx.workDir(), ctx.runId()
        return Map.of("customer", record);
    }
}
```

- **Registration:** add the class to `FirstPartyPackages.ALL` (built-in) or its name to `FirstPartyPackages.OPTIONAL` (optional module).
- **Signing:** the registry signs each toolkit with an Ed25519 publisher key and only installs trusted publishers. Development builds use a generated key; release builds are meant to ship pre-signed JARs.
- **Conditions** (`@Condition`) feed `core/if` and `wait_until`. **Loop iterators** (`LoopIterator`) feed `core/loop`.
- **Errors:** throw `ActionException(code, message, suggestedFix)`. Codes are upper-case and dotted (`GITHUB.NOT_FOUND`). Retryable errors (429, 5xx) are marked so step retries apply.
- **Rules:**
  - every action and input has a description

- ids are `snake_case`
- the word "bot" is not used in user-facing text
- inputs that are secrets are `OoaType.CREDENTIAL`

Today there are **72 toolkits and 875 actions**. The full list is in `docs/COMMAND_CATALOG.md`.

5.3 Runner engine

`Runner.builder(registry).sink(eventSink).workDir(dir).service("vault", vault)...build().run(flow, inputs, options)`

1. **Validate:** `FlowValidator`, plus JSON Schema when adapters-jackson is present.
2. **Create a variable scope** from the inputs. Child agents get a new scope that holds only their inputs and outputs.
3. **Execute steps in order.** For each step:
   - resolve `args`
   - fetch credentials just in time from the `vault` service
   - call the action
   - apply `out`
   - emit events
4. **Control flow:** if/else, loops (plain or iterator-driven), try/catch/finally, parallel, break and continue, and retries with backoff.
5. **Run controls** through `RunOptions`: `cancel` and `pause` flags, `maxRunSeconds`, and a `Debugger` hook. Studio's debugger implements step into, step over, step out, breakpoints and variable edits through that hook.
6. **Events** (`RunEvent`):
   - `run.started`, `run.paused`, `run.resumed`, `run.finished`
   - `step.started`, `step.finished`, `step.failed`, `step.retry`
   - `variable.set`, `log`

They stream to Studio's run panel or, through the One Runner, to Agent Runs in the Command Center. Secret values are masked.

1. **Result** (`RunResult`): status (success, failed or stopped), outputs, error (code, message, step id, suggested fix) and the number of steps executed.

Services the engine looks up (provided by Studio or the One Runner):

- `vault`
- `repository` (other agents)
- `bot_invoker` (child agents)
- `global_values`
- `model_connections`

---

6. Runtime flows

6.1 Building and testing in Studio

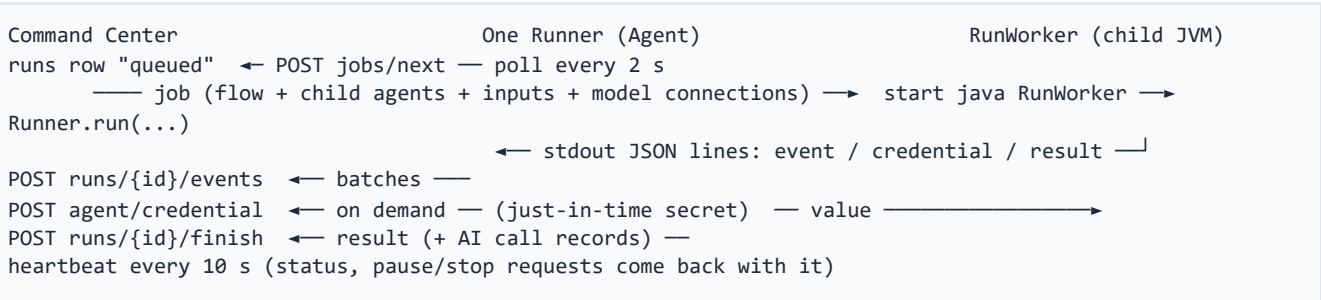
```
studio.js ──HTTP──► StudioServer ──► Workspace (files in My Workspace)
                                └──────────► Runner (same JVM) ──events──► polled by studio.js
```

- **Access:** Studio binds to **127.0.0.1 only**. It refuses non-loopback callers, checks the `Host` header and rejects cross-site `Origin`. When mounted inside `ooa server`, it reuses the browser's Command Center sign-in.
- **API (`/studio/api/...`):** `tree`, `item`, `bot`, `validate`, `run`, `runs`, `packages`, `builtins`, `search`, `deps`, `record`, `capture`, `clipboard`, `connections`, `connectors`, `triggers`, `assistant/build`, `info`. Command Center link: `cr/connect`, `cr/disconnect`, `cr/status`, `cr/repo`, `cr/checkin`, `cr/checkout`, `cr/undo-checkout`, `cr/versions`.
- **Debug runs** use `StudioDebugger`, which pauses the runner thread before a step and resumes it on the UI's commands.

6.2 Check-in to the Command Center

1. Studio sends `POST /api/repo/checkin` with the local file's JSON.
2. The Command Center validates it:
    - agents: Flow validation, and every toolkit used must be switched on
    - workflows: stage checks
    - forms: field checks
3. It writes a new row in `bot_versions` (content, SHA-256, author, comment) and updates `bots`: latest version, optionally the live production version, and the check-out lock.
4. Check-out locks the agent for one person. Runs, schedules, triggers, queues and workflows always use the **live production version**.

6.3 Running an agent on a One Runner



- **Runner sign-in:** `X-OOA-Device` and `X-OOA-Device-Secret` headers, issued at registration with a one-time registration key.
- **Isolation:** each run gets its own JVM (`RunWorker`). A crashing agent never stops the runner. Logs go to `OOA Agent Workspace\logs`.
- **Secrets:** a credential is decrypted only for a running job, only if the run's requester is a consumer of that vault folder, and every read is audited.
- **Picking a runner:** a runner pool picks its least busy online runner. Priority orders the runner's queue.
- **Lost runners:** "online" means a heartbeat within the last 60 s. Runs on a runner that went quiet are marked lost (`AGENT.LOST`).
- **When a run finishes**, `finish()` also:
    - updates work-queue items (retry until max attempts)
    - stores AI call records for governance
    - continues workflows
    - posts the optional callback URL (3 attempts)

6.4 Automatic starts

Source	Where evaluated	Implementation
Schedules (every N min, daily, weekly; time zone; holiday calendar; pool)	Command Center clock, every 15 s	`ControlRoom.tickSchedules`
Work queues (one item per run, SLA, retries)	Command Center clock, every 3 s	`Platform.dispatch`, `itemFinished`
Workflows (stage engine, waits for runs and people)	Command Center clock, every 5 s, plus events	`Workflows.advance`, `onRunFinished`, `complete`
Runner triggers: file/folder, window, process, service, hotkey	On the runner, 2 s polling	`LocalTriggers` → `POST agent/trigger-fired`
Server triggers: IMAP, Outlook (Graph), ServiceNow, universal HTTP poller	Command Center, each trigger's interval (at least 15 s)	`TriggerPollers` + `ImapMailSource`, `GraphMailSource`, `PollHttp`
Webhooks	`POST /api/hooks/{id}` with the `X-OOA-Secret` header	`ControlRoomServer`
API callers and AI assistants	`POST /api/runs` with an API key; `POST /api/mcp`	`ControlRoomServer`, `McpServer`

6.5 One Workflow and the task inbox

A workflow (`*.process.json`) is a list of **stages**: `bot` (agent task), `form` (human task), `approval`, `decision` and `notify`. `Workflows` keeps a `workflow_runs` row for each run, holding its status (running, waiting, completed, failed or cancelled), the current stage, its data and a history.

- Agent tasks start runs with source `workflow:<id>`.
- Human tasks and approvals become `human_tasks` rows that appear in the console's **Task inbox**, filtered by `user:`/`role:` assignee. People claim them and complete them; answers are checked against the form.
- Results merge into the workflow data (also as `$form.x`) and the engine moves to the next stage.

7. Data model (Command Center database)

Schema migrations are versioned in `Db.java` (`SCHEMA_V1`, `SCHEMA_V2`, `SCHEMA_V3`) and recorded in `schema_version`. They run automatically at start.

Area	Tables
People and access	`users` (PBKDF2 hash, role, lockout, SSO subject), `sessions` (token hashes), `roles` (custom), `api_keys` (hashes)
Agent repository	`bots` (path, latest and live production version, lock, minutes saved), `bot_versions` (JSON content, SHA-256, author, comment)
Runners and runs	`devices`, `registration_keys`, `device_pools`, `runs` (status, inputs, outputs, error, priority, callback, queue item), `run_events`
Automation	`schedules`, `triggers` (config JSON, fire count, last fired, last error, cursor), `holiday_calendars`, `queues`, `queue_items`, `workflow_runs`, `human_tasks`

Area	Tables
Secrets and settings	`lockers` (vault folders, consumers, owners, external vault link), `credentials` (AES-GCM ciphertext), `global_values` (shared values), `settings`, `model_connections`, `toolkit_settings`
Oversight	`audit` (hash chain: `prev_hash`, `hash`), `ai_calls` (model, tokens, masking, tools, prompt fingerprint)

New tables or columns go into a **new** `SCHEMA_Vn` array. Never edit an earlier version that may already be applied to someone's database.

8. Security model

Concern	How
Passwords	PBKDF2-HMAC-SHA256, 210,000 iterations, random salt; 12+ characters with letters and digits; 5 failures lock for 15 min; first sign-in and resets force a change
Sessions	Random token, stored as SHA-256; `HttpOnly; SameSite=Strict` cookie `ooa_cr`, 8 h sliding; every browser change needs the `X-OOA-Request: 1` header (CSRF)
API	`Authorization: Bearer <session token or ooak_ API key>`; API keys are stored as hashes, can expire and be revoked
Single sign-on	OpenID Connect authorization code with PKCE; ID token signature (RS256/384/512, ES256) checked against the provider's JWKS, plus issuer, audience, expiry and nonce
Permissions	`Rbac`: built-in roles (admin, bot_creator, bot_runner, viewer, shown as Administrator, Agent builder, Agent operator, Viewer) plus custom roles; checks like `require(user, "queues.manage")` in every service method
Vault	AES-256-GCM. Master key from `OOA_MASTER_KEY` (Base64, 32 bytes) or `master.key` in the data folder. Each value is bound to its `folder/credential/attribute` name. Owners and consumers per folder; values never returned to the UI
External vaults	HashiCorp KV v2, Azure Key Vault (client credentials), AWS Secrets Manager (SigV4), CyberArk CCP, read live per use; the sign-in secret is encrypted in the vault
Audit	Append-only, hash-chained records (`Audit.java`); "Verify integrity" recomputes the chain
Secrets in runs	`SecureString` values are masked in logs and events; toolkits must not print
AI	PII masking before model calls; governance records store tokens, model and a SHA-256 fingerprint of the prompt, never the prompt
Studio	Loopback only, host and origin checks
Web pages	Strict Content-Security-Policy (`script-src 'self'`), `X-Frame-Options: DENY`
Supply chain	Toolkits signed with Ed25519; publisher allow-list in `PackageRegistry`

9. Configuration

9.1 Command line (`ooa`, main class `com.oneofficeautomation.cli.Ooa`)

Run through Gradle from `platform/`: `.\gradlew.bat :cli:run --args="<command> <options>"`. You can also build a distribution with `.\gradlew.bat :cli:installDist` and run `cli\build\install\ooa\bin\ooa.bat`.

Command	Options	Purpose
`server`	`--port 8720`, `--workspace DIR`, `--data DIR`, `--no-agent`, `--no-browser`	Everything on one port (development and single-PC use)
`control-room`	`--port 8720`, `--host 127.0.0.1|0.0.0.0`, `--data DIR`, `--db-url URL --db-user U --db-password-env VAR`	Command Center only (server use)
`studio`	`--workspace DIR`, `--port 8710`	Studio only
`agent register` / `agent start`	`--url URL --key KEY [--name N]` / `[--workspace DIR]`	One Runner on another machine
`reset-admin`	`[--user NAME] [--data DIR]`	Forgotten password (server stopped)
`run`, `validate`, `manifest`, `conformance`		Developer tools: run or validate a Flow file, print toolkit manifests, run the conformance cases

9.2 Folders and environment

Item	Default	Override
Command Center data (H2 database, `master.key`)	`%USERPROFILE%\OOA Control Room` when it holds a database, else `%USERPROFILE%\OOA Command Center`	`--data`
My Workspace (Studio files)	`%USERPROFILE%\OOA Workspace`	`--workspace`
One Runner working folder and logs	`%USERPROFILE%\OOA Agent Workspace`	`--workspace` on `agent start`
Vault master key	key file in the data folder	`OOA_MASTER_KEY` (recommended for production)
PostgreSQL	not used	`--db-url jdbc:postgresql://host:5432/ooa --db-user ooa --db-password-env OOA_DB_PASSWORD`
Optional toolkit modules	all six	`platform/gradle.properties` → `ooa.optionalModules`
AI approval timeout	300 s	`-Dooa.approval.timeoutSeconds=N`
Headless (no dialogs)	off	`-Djava.awt.headless=true`

9.3 Settings in the product (Administration > Settings, table `settings`)

- `announcement`
- `inactive_user_days`, `session_hours`, `max_upload_mb`
- `queue_poll_seconds`, `default_minutes_saved`
- `public_url`
- `sso_issuer`, `sso_client_id`, `sso_client_secret_ref`, `sso_default_role`, `sso_label`
- `mcp_agents`, `mcp_runner`, `mcp_timeout_seconds`

They are defined with defaults in `Platform.DEFAULTS`. Adding a setting means adding a default there; it then appears in the API at once.

10. Managing the platform

Task	Where
Start or stop on one PC	`platform\scripts\start-ooa.bat`; Ctrl+C stops (see `docs/RUN_LOCALLY.md`)
Users, roles, API keys, SSO	Command Center > Users, Administration > Roles / Settings, top bar > API keys
Runners	Runners > Register a runner (one-time key); runner pools tab; `scripts\install-agent-task.ps1` starts a runner at sign-in
Toolkits on or off	Administration > Toolkits (shows which agents use each)
AI models	Administration > One Model Connections; Studio > One Model Connections for local runs
Secrets	One Agent Vault (folders, consumers, owners, external vault link, Test reading)
Monitoring	Home dashboard, Agent Runs (Live/History, CSV export), Work queues, One Workflow, Triggers monitoring column, AI governance, One Audit Trail (Verify integrity)
Backup	The data folder (`controlroom.mv.db` + `master.key`) and My Workspace; for PostgreSQL, the database plus `OOA_MASTER_KEY`
Upgrade	`git pull`, restart; migrations run automatically
Recovery	`reset-admin`; the audit trail records it

Production notes:

- Put an HTTPS reverse proxy in front before using `--host 0.0.0.0`.
- Use PostgreSQL and `OOA_MASTER_KEY`.
- Set `public_url` for SSO.
- Live views poll every 2 to 5 s; a push channel is a planned improvement.

11. Testing

Suite	How to run (from `platform`, PowerShell)	Covers
All Java tests	`.\gradlew.bat test` (add `$env:JAVA_TOOL_OPTIONS='-Djava.awt.headless=true'` on a desktop)	314 tests: runner, every toolkit against local fake services, Command Center end to end (`ControlRoomE2ETest`, `PlatformTest`, `EnterpriseFeaturesTest`, `UnifiedServerTest`, trigger tests), debugger, architecture rules
One module or class	`.\gradlew.bat :packages:test --tests "*GitHubPackageTest*"`	
Studio UI	`cli/src/test/ui/studio.ui.test.js` with Node + jsdom against a running `ooa studio`	131 checks
Console UI	`control-room/src/test/ui/console.ui.test.js` with jsdom against a running `ooa control-room` (env `CR_URL`, `CR_ADMIN_PASSWORD`)	58 checks

Conventions:

- Cloud toolkits are tested with a local fake HTTP server (`ConnectorsTest.Fake`, `ToolkitTestSupport`, `RestToolkitTestSupport`). Each test checks the method, path, query, headers and body sent, the outputs, the error codes, and that no secret appears in logs.
- Run Java tests from **PowerShell or cmd**, not Git Bash. The PGP test needs a Windows gpg.

---

12. How to build new components

12.1 A new toolkit (most common)

1. Create `platform/packages/src/main/java/com/oneofficeautomation/packages/<Name>Package.java`. Use an optional module instead if it needs a heavy library: create `platform/packages-<x>`, add it to `ooa.optionalModules`, and give it its own `build.gradle.kts`.
2. Annotate it as in 5.2. For a web API, copy the pattern of `JiraPackage` or `GitHubPackage`:
   - use `HttpJson`
   - take a base URL input
   - take a Credential token
   - follow paging
   - map errors to `NAME.CODE` and mark 429/5xx retryable
3. Register it in `FirstPartyPackages` (`ALL`, or `OPTIONAL` by class name).
4. Write `<Name>PackageTest` with the fake server, and run `.\gradlew.bat :packages:test --tests "*<Name>*"` plus `ArchitectureTest`.
5. Add a row to `docs/COMMAND_CATALOG.md`. If its category is new, add it to `PKG_CAT` in `studio.js`.
6. Nothing else is needed: Studio's action list, validation, the AI assistant and the Command Center's Toolkits page all read the manifest.

12.2 A new Command Center feature

1. **Data:** add a `SCHEMA_V4` array in `Db.java` and a `current < 4` step in `migrate()`.
2. **Logic:** add a service method in `Platform.java` (or a new class like `Workflows`). Start with `cr.require(user, "<permission>", "...")`, use `db.query/one/update` with `?` placeholders, and record `cr.audit.record(...)`.
3. **Permission:** add it to `Rbac` (and to `ADMIN_ONLY` if needed).
4. **API:** add a `case "<name>" ->` route in `ControlRoomServer.api(...)`.
5. **UI:**
   - add a page to `PAGES` and `VIEWS` in `cr.js`
   - build it with the helpers `h()`, `header()`, `dataTable()`, `modal()`, `openDrawer()`, `tabStrip()`, `kv()`, `pill()`
   - add styles to `cr.css`
6. **Tests:** a Java end-to-end test (see `PlatformTest`) and checks in `console.ui.test.js`.

12.3 A new Studio capability

- **API:** add a route in `StudioServer`. Keep it loopback-only, which the server already enforces.
- **UI:** add it to `studio.js`, using the same `h()` helper, dialogs and `api()` wrapper. Check the file with `node --check`.

- **Engine changes:** add an interface or hook in `runner` (as `Runner.Debugger` was), never Studio code in the runner.

12.4 A new trigger kind

- **On the runner:** add a watcher to `LocalTriggers` (agent module) and its kind to `TriggerKinds`.
- **On the server:** add a source to `TriggerPollers` and keep the cursor in `triggers.last_hash`.
- **In the console:** add its fields to `triggerDialog` in `cr.js`.
- **Input:** pass the event to the agent as `trigger_<kind>` plus `trigger_event`.

12.5 A new AI model provider

- Add a branch in `LlmClient` (request format and response parsing).
- Add the provider value to the enum lists in `GenAiPackage`, `AiAgentPackage` and `AiSkillsPackage`, to `Platform.PROVIDERS`, to `StudioServer.PROVIDERS`, and to `AI_PROVIDERS` in `studio.js`.

12.6 Rules of thumb

- Keep the dependency direction of section 4. The architecture test will tell you when you break it.
- **Secrets:**
  - Never take a secret as plain text: use Credential inputs or vault references.
  - Never log a secret.
  - Never return a stored secret to the browser.
- Every user-facing string uses the One naming (agent, One Runner, Command Center…), never "bot".
- No new dependency without need. If one is needed, it must be Apache-2.0, MIT or similar, and it goes in an optional module.
- Record each larger change's test results in `docs/VALIDATION_REPORT.md`, as the earlier parts do.

---

13. Known gaps and planned work

Gap	Note
Class-loader isolation per toolkit version	Plan: PF4J; today all toolkits share one class path
Compile-time manifests	Plan: an annotation processor instead of runtime introspection
Push channel for live views	Today polling (2 to 5 s)
Signed custom toolkit upload	Toolkit manager lists and switches toolkits; upload not built
Git sync of the agent repository	Not built
SAP BAPI	Needs the customer's SAP JCo library
macOS/Linux runners	Windows runner first; most toolkits are cross-platform
Live-tenant checks	Cloud toolkits, SSO, external vaults and mail triggers are tested against simulators only

Related documents:

- `docs/RUN_LOCALLY.md`: start, sign in, reset a password

- `docs/CONTROL_ROOM_AND_AGENT.md`: Command Center and One Runner user guide
- `docs/COMMAND_CATALOG.md`: every toolkit
- `docs/TRACEABILITY.md`: requirements
- `docs/VALIDATION_REPORT.md`: test results per part
- `docs/research/AA_GAP_ANALYSIS.md`: feature comparison and backlog